Authorization Code Grant

less than a 15 minute read

On this page

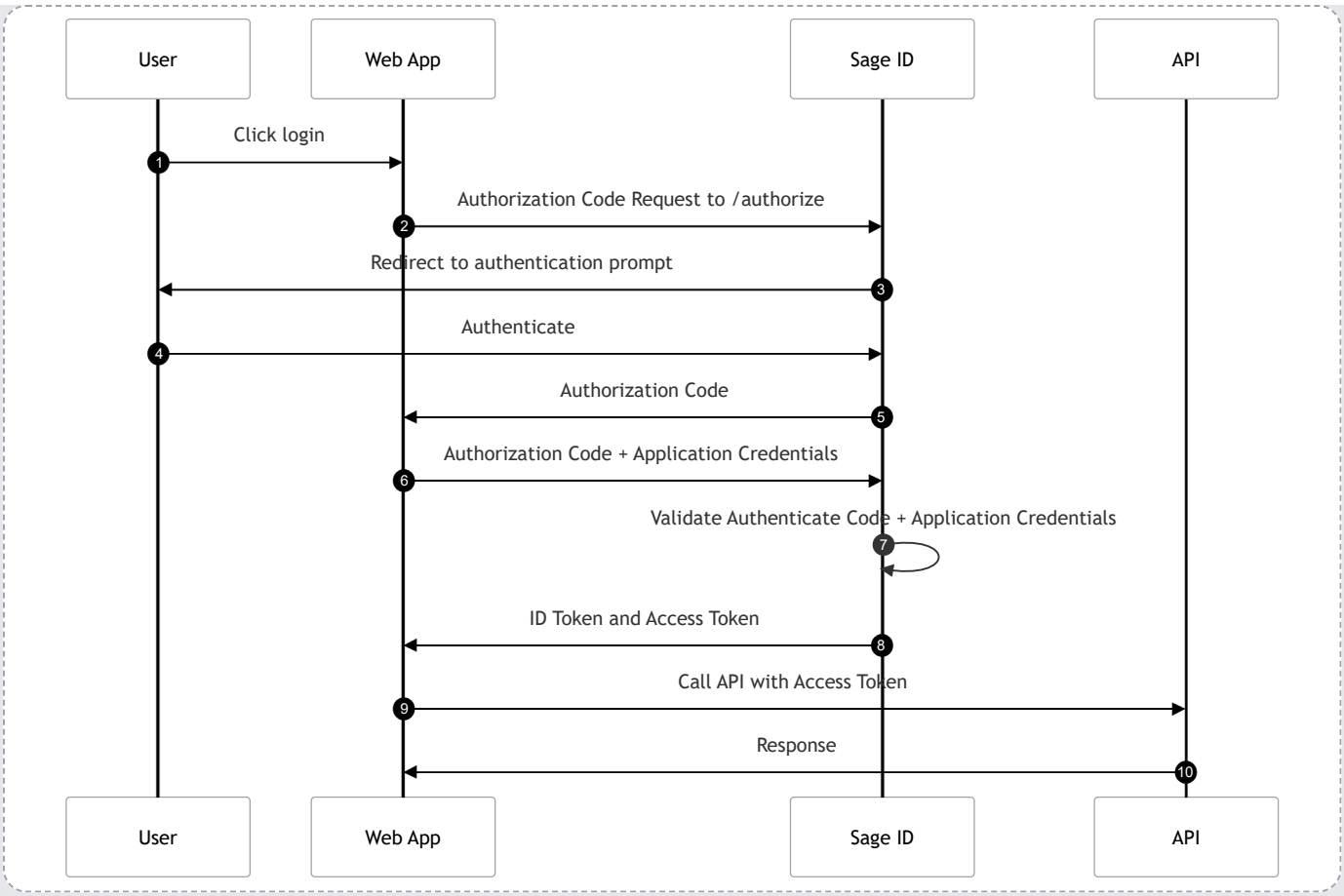
Overview of the flow

The **Authorization Code OAuth 2.0 grant** (defined in RFC 6749, section 4.1) flow involves **exchanging** an **Authorization Code** for an **Access Token**, and optionally an **ID Token** and a **Refresh Token**.

Warning

It must be used with a **confidential** client.

- Only use with **Regular Web Applications**, where the Client Secret can be safely stored.
- In **Single Page Applications** the integrity of the Client Secret cannot be maintained.



1. User selects Login within application.

2. Redirect user to Sage ID (`/authorize` endpoint).

3. Redirect user to login and authentication prompt.

4. User **authenticates**.

5. Sage ID redirects the user to the web app (specifically to the `redirect_uri`, as specified in the `/authorize` request) with an **Authorization Code** in the querystring (code).

6. The web app sends the **Authorization Code**, application's client ID, and client secret, to Sage ID (`/oauth/token` endpoint).

7. Sage ID **verifies** authorization code, application's client ID, and application's client secret.

8. Sage ID responds with an **ID Token** and **Access Token** (and optionally, a **Refresh Token**).

9. Application can use the Access Token to call the API.

10. API responds with requested data.

> In OAuth 2.0 terms, the web app is the application, the end user the Resource Owner, the API the Resource Server, and Sage ID the Authorization Server.

Pre-requisites

Your **API** and **client applications must be correctly configured** in Sage ID using **Developer Self Service**.

The application owner needs to perform the following configurations:

1. Select **Web Application** as the client type.
2. Make sure that your application **grant types** include **Authorization Code**.
3. If you want to be able to use **refresh tokens**, make sure the Application's **grant types** include refresh token. More information on Tokens is available in the Tokens section.
4. Provide a **callback URL**.

The API owner need toperform the following configurations:

1. Enable `offline_access` if refresh tokens are needed.

> **Warning**
>
> Rotating refresh tokens are required.

Execute an Authorization Code Grant Flow

The Authorization Code grant is used by Regular Web Applications in order to access an API and authenticate users.

1. Call the Authorization URL

First, the user should be sent to the authorization URL:

txt Copy

```txt
https://id.sage.com/authorize?
audience=YOUR_API_AUDIENCE&
scope=YOUR_SCOPE&
response_type=code&
```

```
client_id=YOUR_CLIENT_ID&
redirect_uri=https://YOUR_APP/callback&
state=YOUR_OPAQUE_VALUE
```

For testing, our Shadow environment can be used:
`https://id-shadow.sage.com/authorize?`

This endpoint supports the following parameters:

Parameter	Description
`audience`	**Unique identifier** of the API, only needed for Access Tokens.
`scope`	The scopes that the authorization is requested for. Must be separated by a space. Any standard OIDC scopes can be requested (such as `profile` and `email`), custom claims, or any scopes supported by the API. Include `offline_access` to get a Refresh Token.
`response_type`	The credential that Sage ID will return. For this flow, the value **must** be **code**.
`client_id`	The application's Client ID.
`state`	An opaque value the application adds to the initial request that Sage ID includes when redirecting back to the application. This value **must be used** by the application **to prevent CSRF**(Cross Site Request Forgery) **attacks**.
`redirect_uri`	The URL to where the browser will be **redirected after authentication** of the user. The `Authorization Code` will be available in the code URL parameter. This URL must be specified as a valid callback URL when you provide the API configuration.
`login_hint`	An `email address` or a base64 and URL encoded **JSON object** may be provided that can be pre-populated on the authentication screen. See Login hints.

html Copy

```
<a href="https://id.sage.com/authorize?scope=appointments%20contacts&audience=sci:appoi
  Sign In
</a>
```

The authorization code is included in the query string parameter after `code=` :

html Copy

```html
HTTP/1.1 302 Found
Location: {https://YOUR_APP/callback&}?code={authorizationCode}&state=xyzABC123
```

2. Exchange the Authorization Code for a Token

The `Authorization Code` must be exchanged for an **ID Token** and/or **Access Token**, and potentially a **Refresh Token**. Create a **POST** request to the Token URL (`/oauth/token`) using the **Authorization Code** (`code`):

c# Copy

```c#
var client = new RestClient("https://id.sage.com/oauth/token");
var request = new RestRequest(Method.POST);
request.AddHeader("content-type", "application/x-www-form-urlencoded");
request.AddParameter("application/x-www-form-urlencoded",
"grant_type=authorization_code&
client_id=YOUR_CLIENT_ID&
client_secret=YOUR_CLIENT_SECRET&
code=YOUR_AUTHORIZATION_CODE&
redirect_uri=https://YOUR_APP/callback", ParameterType.RequestBody);
IRestResponse response = client.Execute(request);
```

Where:

- `grant_type` : **Must** be `authorization_code` .
- `client_id` : Your application's Client ID.
- `client_secret` : Your application's Client Secret.
- `code` : The Authorization Code received.
- `redirect_uri` : The URL must match exactly the `redirect_uri` passed to `/authorize` .

Each parameter must be **URL encoded** according to RFC1866 section 8.2.1. Spaces must be encoded as a + (plus) character.

The response contains the `access_token`, `refresh_token`, `id_token`, and `token_type` values, for example:

javascript Copy

```javascript
{
  "access_token": "eyJz93a...k4laUWw",
  "refresh_token": "GEbRxBN...edjnXbL",
  "id_token": "eyJ0XAi...4faeEoQ",
  "token_type": "Bearer"
}
```

- The **Access Token** can be used to call an API.
- The **ID Token** can be used to authenticate the user.
- The **Refresh Token** can be used to get a new Access Token after it expired.

The `refresh_token` will only be present in the response if `offline_access` is included in the scope and the API is configured to support this option.

3. Call the API

Calls can be made to the API by passing the `access_token` as a Bearer Token in the `Authorization` header:

c# Copy

```csharp
var client = new RestClient("https://someapi.com/api");
var request = new RestRequest(Method.GET);
request.AddHeader("authorization", "Bearer ACCESS_TOKEN");
request.AddHeader("content-type", "application/json");
IRestResponse response = client.Execute(request);
```

See how to **Verify the token**.

Previous: Client... Next: Authorization Code Flow with Proof Key for Code...

Was this page helpful?

Yes No

Report issue